Competitive Programming

Section 43 Editorial: String Algorithms I

Competitive Programming Course

2026-10-20

Table of contents

1 Editorial overview **3**
- 1.1 Problem-specific proof map . . . 3
- 1.2 Detailed solution guide . . . 3

2 A. KMP Occurrences **5**
- 2.1 Restatement . . . 5
- 2.2 Observations and algorithm . . . 5
- 2.3 Correctness proof . . . 5
- 2.4 Complexity . . . 5
- 2.5 Full C++ solution . . . 5
- 2.6 Full Python solution . . . 6
- 2.7 Tests that matter . . . 7

3 B. All Borders **8**
- 3.1 Restatement . . . 8
- 3.2 Observations and algorithm . . . 8
- 3.3 Correctness proof . . . 8
- 3.4 Complexity . . . 8
- 3.5 Full C++ solution . . . 8
- 3.6 Full Python solution . . . 9
- 3.7 Tests that matter . . . 10

4 C. Minimal Period **11**
- 4.1 Restatement . . . 11
- 4.2 Observations and algorithm . . . 11
- 4.3 Correctness proof . . . 11
- 4.4 Complexity . . . 11
- 4.5 Full C++ solution . . . 11
- 4.6 Full Python solution . . . 12
- 4.7 Tests that matter . . . 12

5 What to carry forward **13**

6 Testing discussion **14**

7 D. Prefix Frequency **15**
- 7.1 Restatement and traps . . . 15
- 7.2 Observation and trace . . . 15
- 7.3 Correctness proof . . . 16
- 7.4 Complexity . . . 16
- 7.5 Full C++ solution . . . 16
- 7.6 Full Python solution . . . 17
- 7.7 Tests and stress testing . . . 17

1 Editorial overview

1.1 Problem-specific proof map

A. KMP Occurrences. The prefix function records the longest matched prefix after every text position; falling through failure links never discards a possible occurrence. Each character advances or falls only amortized constant times, so matching is `O(|text|+|pattern|)`.

B. All Borders. Repeatedly following `pi[length-1]` lists exactly the proper prefix lengths that are also suffixes, from longest to shortest; reverse the collected list for increasing output.

C. Minimal Period. If `p=n-pi[n-1]` divides `n`, copies of the length-`p` prefix cover the string; otherwise no shorter border-compatible block can tile it, so the period is `n`.

1.2 Detailed solution guide

A. KMP Occurrences. Build the combined string `pattern + '#' + text`, where `#` is a separator that cannot appear in either input. The prefix function value `pi[i]` is the length of the longest prefix of the combined string ending at `i`. When `pi[i] == m`, an occurrence of the pattern has just ended. The end is at combined index `i`, the text starts at combined index `m + 1`, so the 1-based start position in the text is `i - 2*m + 1`. Overlapping matches are handled automatically because the prefix function falls back through borders instead of resetting to zero.

B. All Borders. A border of `s` is both a prefix and a suffix. The longest proper border has length `pi[n-1]`. If a string of length `k` is a border, then its own longest proper border has length `pi[k-1]`, and that is also a border of the original string. Repeating this prefix-link walk lists every border from longest to shortest; reversing gives increasing order. No border is skipped, because every shorter border of `s` must be a border of the longest border that contains it.

C. Minimal Period. If the longest border has length `b`, then the first candidate period is `p = n - b`: the suffix that repeats the prefix begins after `p` characters. This candidate is valid only when `n % p == 0`; otherwise the repeated blocks do not tile the string and the answer is `n`. This is why strings such as `ababab` return `2`, while `ababa` returns `5`.

Common pitfalls. The separator must be absent from the alphabet, positions for A are 1-based, and B should not print the full length because a border is proper in this exercise.

The prefix function is the only real tool in this section. It is computed once in linear time and then interpreted in three ways.

- A builds `pattern + separator + text`; whenever `pi[i]` equals the pattern length, an occurrence ends at `i`.
- B follows prefix links from `pi[n-1]` to list every proper border of the full string.
- C uses the longest border to derive the candidate period `n - pi[n-1]`, then checks divisibility.

The important proof idea is that prefix links enumerate exactly the shorter borders, so no possible border or period is skipped.

2 A. KMP Occurrences

2.1 Restatement

Print the number and 1-based positions of all occurrences of pattern in text.

2.2 Observations and algorithm

Build `pattern + '#' + text` and its prefix function. If pattern length is `m`, `pi[i]==m` means a match ends at combined index `i`, whose 1-based text start is `i-2*m+1`. For `aba` in `abacaba`, this produces starts 1 and 5, including overlaps. Compute pi once, then scan its values.

2.3 Correctness proof

`pi[i]` is the longest pattern-prefix ending at i; consequently value m is exactly a complete occurrence. Failure links retain every shorter possible border, so no occurrence is skipped. QED.

2.4 Complexity

Time and memory are `O(|pattern|+|text|)`.

2.5 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;

vector<int> pi(const string& s) {
    vector<int> p(s.size());
    for (int i = 1; i < (int)s.size(); i++) {
        int j = p[i - 1];
        while (j && s[i] != s[j]) {
            j = p[j - 1];
        }
        if (s[i] == s[j]) {
            j++;
        }
        p[i] = j;
```

```
    }
    return p;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string pat, text;
    if (!(cin >> pat >> text)) {
        return 0;
    }
    string s = pat + "#" + text;
    auto p = pi(s);
    vector<int> ans;
    for (int i = 0; i < (int)p.size(); i++) {
        if (p[i] == (int)pat.size()) {
            ans.push_back(i - 2 * (int)pat.size() + 1);
        }
    }

    cout << ans.size() << '\n';
    for (int i = 0; i < (int)ans.size(); i++) {
        if (i) {
            cout << ' ';
        }
        cout << ans[i];
    }
    cout << '\n';
}
```

2.6 Full Python solution

```
import sys


def pi(s):
    p = [0] * len(s)
    for i in range(1, len(s)):
        j = p[i - 1]
        while j and s[i] != s[j]:
            j = p[j - 1]
        if s[i] == s[j]:
            j += 1
        p[i] = j
    return p
```

```
def main():
    lines = sys.stdin.read().splitlines()
    pat = lines[0]
    text = lines[1]
    s = pat + "#" + text
    p = pi(s)
    ans = []
    for i, value in enumerate(p):
        if value == len(pat):
            ans.append(i - 2 * len(pat) + 1)

    print(len(ans))
    print(" ".join(map(str, ans)))


if __name__ == "__main__":
    main()
```

2.7 Tests that matter

Test pattern longer than text, equal strings, no match, and overlapping `ana` in `anana`. The separator must be outside the input alphabet.

3 B. All Borders

3.1 Restatement

Print all proper border lengths of a string in increasing order.

3.2 Observations and algorithm

The longest proper border is `pi[n-1]`. Repeatedly set `k=pi[k-1]` to follow the borders of that border; collect lengths and reverse for increasing output. For `ababa`, this visits 3 then 1.

3.3 Correctness proof

Every shorter border of the string is a border of its longest border, and every prefix link is a border by definition, so the walk lists exactly all borders. QED.

3.4 Complexity

Prefix construction and the link walk are `O(n)` time and `O(n)` memory.

3.5 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;

vector<int> pi(const string& s) {
    vector<int> p(s.size());
    for (int i = 1; i < (int)s.size(); i++) {
        int j = p[i - 1];
        while (j && s[i] != s[j]) {
            j = p[j - 1];
        }
        if (s[i] == s[j]) {
            j++;
        }
        p[i] = j;
    }
    return p;
```

```cpp
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    if (!(cin >> s)) {
        return 0;
    }
    auto p = pi(s);
    vector<int> ans;
    for (int k = p.back(); k; k = p[k - 1]) {
        ans.push_back(k);
    }
    reverse(ans.begin(), ans.end());

    for (int i = 0; i < (int)ans.size(); i++) {
        if (i) {
            cout << ' ';
        }
        cout << ans[i];
    }
    cout << '\n';
}
```

3.6 Full Python solution

```python
import sys


def pi(s):
    p = [0] * len(s)
    for i in range(1, len(s)):
        j = p[i - 1]
        while j and s[i] != s[j]:
            j = p[j - 1]
        if s[i] == s[j]:
            j += 1
        p[i] = j
    return p


def main():
    s = sys.stdin.readline().strip()
    p = pi(s)
    ans = []
    k = p[-1] if s else 0
```

```
    while k:
        ans.append(k)
        k = p[k - 1]
    print(" ".join(map(str, ans[::-1])))


if __name__ == "__main__":
    main()
```

3.7 Tests that matter

Test a singleton, `aaaa`, and a string without a border. Do not output n: this exercise asks only for proper borders.

4 C. Minimal Period

4.1 Restatement

Print the shortest period length if the string is repetitions of a block, otherwise `n`.

4.2 Observations and algorithm

Let `p=n-pi[n-1]`. It is a tiling period only if `n%p==0`; otherwise print n. For `abcabcabc`, p is 3; for `ababa`, candidate p is 2 but answer is 5.

4.3 Correctness proof

A border of length n-p proves equality after shift p. Divisibility proves these shifts cover the whole string as equal blocks; without it, no shorter block tiles the string. QED.

4.4 Complexity

Time is `O(n)` and memory is `O(n)`.

4.5 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;

vector<int> pi(const string& s) {
    vector<int> p(s.size());
    for (int i = 1; i < (int)s.size(); i++) {
        int j = p[i - 1];
        while (j && s[i] != s[j]) {
            j = p[j - 1];
        }
        if (s[i] == s[j]) {
            j++;
        }
        p[i] = j;
    }
    return p;
```

```cpp
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    if (!(cin >> s)) {
        return 0;
    }
    int n = s.size();
    auto p = pi(s);
    int k = n - p.back();
    cout << (n % k == 0 ? k : n) << '\n';
}
```

4.6 Full Python solution

```python
import sys


def pi(s):
    p = [0] * len(s)
    for i in range(1, len(s)):
        j = p[i - 1]
        while j and s[i] != s[j]:
            j = p[j - 1]
        if s[i] == s[j]:
            j += 1
        p[i] = j
    return p


def main():
    s = sys.stdin.readline().strip()
    n = len(s)
    k = n - pi(s)[-1]
    print(k if n % k == 0 else n)


if __name__ == "__main__":
    main()
```

4.7 Tests that matter

Test one character, all one character, a prime-length nonperiodic string, and a near-repeat with a different final character.

5 What to carry forward

Advanced algorithms become manageable when you state the invariant first, then code only the operations needed to preserve it.

6 Testing discussion

For random strings of length at most 30, compare KMP positions with direct substring comparison, borders with a loop over lengths, and the period with a loop over every divisor of the length. Run:

```
CP_TARGET=solution python3 sections/43_string_algorithms_i/check.py
```

7 D. Prefix Frequency

7.1 Restatement and traps

For every prefix length `L`, count all substring occurrences of that prefix, including overlaps. With `n <= 200000`, comparing every prefix at every start is quadratic. The whole string itself occurs once, and a singleton prefix can occur many times.

7.2 Observation and trace

Let `pi[i]` be the longest proper prefix ending at position `i`. Every prefix ending at `i` contributes one occurrence to length `pi[i]`; that occurrence also belongs to every border of that prefix. Count direct `pi` values, then propagate counts from long lengths to their failure link:

```
for x in pi: count[x]++
for length from n-1 down to 1:
    count[pi[length-1]] += count[length]
answer[length] = count[length] + 1
```

The final `+1` counts the prefix occurrence beginning at position zero. Here is the complete small trace for `ababa`:

```
index i:       0 1 2 3 4
character:     a b a b a
pi[i]:         0 0 1 2 3
```

Directly incrementing the nonzero `pi` values gives:

```
length L:      1 2 3 4 5
direct count:  1 1 1 0 0
```

Now propagate in decreasing length. Length `3` passes its one occurrence to its failure link `pi[2]=1`; the resulting relevant counts are:

```
after L=3:     count[1]=2, count[2]=1, count[3]=1
```

Lengths `2` and `1` only pass counts to the zero-length sentinel. Adding the initial occurrence to every positive length gives `3 2 2 1 1`. In particular, the occurrence of `aba` also contributes to its border `a`; that is exactly what the failure-link propagation preserves.

7.3 Correctness proof

Lemma. After propagation, `count[L]` is the number of non-initial occurrences of the prefix of length `L`.

Proof. Every occurrence ending at a position first contributes to its longest proper prefix match. Its failure-link chain lists exactly all shorter prefixes that also end there. Processing links from longer to shorter passes each occurrence to every such border once. □

Theorem. `count[L]+1` is the total occurrence count of prefix length `L`.

Proof. The lemma counts all occurrences ending after the initial prefix; the unmatched initial occurrence contributes the one added explicitly. □

7.4 Complexity

The prefix function and failure-link propagation are both `O(n)` time and use `O(n)` memory.

7.5 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;

vector<int> pi(const string& s) {
    vector<int> p(s.size());
    for (int i = 1; i < (int)s.size(); i++) {
        int j = p[i - 1];
        while (j && s[i] != s[j]) {
            j = p[j - 1];
        }
        if (s[i] == s[j]) {
            j++;
        }
        p[i] = j;
    }
    return p;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    cin >> s;
    int n = s.size();
    auto p = pi(s);
    vector<int> cnt(n + 1);

    for (int x : p) {
```

```
        cnt[x]++;
    }
    for (int i = n - 1; i > 0; i--) {
        cnt[p[i - 1]] += cnt[i];
    }
    for (int i = 1; i <= n; i++) {
        cout << cnt[i] + 1 << (i == n ? '\n' : ' ');
    }
}
```

7.6 Full Python solution

```
import sys


def pi(s):
    p = [0] * len(s)
    for i in range(1, len(s)):
        j = p[i - 1]
        while j and s[i] != s[j]:
            j = p[j - 1]
        if s[i] == s[j]:
            j += 1
        p[i] = j
    return p


def main():
    s = sys.stdin.buffer.readline().strip()
    n = len(s)
    p = pi(s)
    cnt = [0] * (n + 1)

    for x in p:
        cnt[x] += 1
    for i in range(n - 1, 0, -1):
        cnt[p[i - 1]] += cnt[i]
    print(*(cnt[i] + 1 for i in range(1, n + 1)))


if __name__ == "__main__":
    main()
```

7.7 Tests and stress testing

Test `a`, `aaaa`, and a string with no repeated prefix. For tiny strings, the oracle directly compares every prefix against every legal substring start; this is independent of prefix links. The scale

case is `200000` copies of `a`, whose answers are `n,n-1,...,1`, exposing quadratic matching.